Hypernova Ticketing: memoria técnica y resultados

Estado documentado: 16 de octubre de 2026 (America/Bogota). Proyecto de prueba técnica para venta de entradas y control de acceso. Este documento distingue resultados automatizados conservados en el repositorio, observaciones manuales del autor y escenarios aún pendientes.

Presentación para el portafolio

Plataforma de ticketing con backend NestJS, PostgreSQL, portal React y aplicación React Native con Expo. Implementa reservas temporales, pagos simulados idempotentes, credenciales Ed25519 rotativas y sincronización de accesos. Una prueba de carga en una VM de GCP confirmó exactamente 50 ventas ante 200 compradores concurrentes, con 150 rechazos por agotamiento y cero errores inesperados. La cámara de un emulador Android leyó una entrada válida. La demostración completa de dos escáneres offline permanece pendiente.

No hay pasarela bancaria real. Los cobros y reembolsos pertenecen a una simulación transaccional. El despliegue en GCP se accede por túnel SSH; no se publica como una URL HTTPS de producción.

Componentes y recorrido de una compra

Componente	Responsabilidad
API NestJS / TypeScript	Identidad, roles, inventario, reservas, pagos simulados, transferencias, accesos y auditoría
PostgreSQL 16	Persistencia, restricciones, bloqueos transaccionales, idempotencia y registro financiero
Portal React	Eventos, zonas, precios, puertas, staff, ventas, reembolsos, incidentes y auditoría
App React Native / Expo	Registro, compra, billetera rotativa y modo escáner; SQLite y SecureStore
Docker Compose	Arranque, secretos aleatorios, migraciones y semilla automática e idempotente

1. El asistente selecciona evento, zona y cantidad. El servidor determina el precio vigente, en centésimos enteros.

2. Una transacción reserva inventario y comprueba el límite por cuenta. La reserva dura 600 segundos por defecto.

3. El pago simulado, el cargo en el ledger y la emisión se confirman juntos. Las claves de idempotencia permiten repetir una solicitud sin otro cobro o emisión.

4. Si una reserva vence, libera cupo. El pago tardío se resuelve mediante devolución simulada sin emitir entrada.

5. La billetera obtiene una credencial del emisor vinculada a la clave pública del titular. El dispositivo firma una prueba de posesión para cada intervalo de 25 segundos.

6. El escáner comprueba firma, tiempo, evento, zona y revocaciones conocidas. Guarda cada intento y su UUID localmente antes de sincronizar.

Garantías y límites del diseño

Inventario: bloqueos de PostgreSQL y restricciones evitan vender o reservar por encima del aforo. El límite por usuario/evento también se serializa entre zonas. La disponibilidad pública se almacena en caché durante 500 ms; la compra siempre vuelve a comprobar el inventario en la base.

Idempotencia: se conserva una clave por usuario y operación con huella de la solicitud. Repetir la misma petición devuelve el resultado; reutilizar la clave con contenido distinto produce conflicto. Una integración bancaria futura necesitaría además coordinación con el proveedor de pagos.

QR: Ed25519 autentica la credencial del emisor y la posesión del titular. Se acepta el intervalo actual y el anterior: un código recién generado puede durar hasta unos 50 segundos; una captura de hace un minuto se

rechaza. No son QR estáticos ni códigos válidos indefinidamente. Todos los clientes y servidor deben usar la misma versión del protocolo.

Offline: se impide la reutilización local con SQLite. Dos puertas completamente aisladas pueden admitir físicamente la misma entrada; al sincronizar se conserva el segundo intento y se genera un incidente, con un único ingreso lógico central. Ningún escáner sin comunicación conoce una revocación posterior a su última sincronización. El sistema no promete impedir físicamente la copia entre puertas desconectadas.

Seguridad: roles y propiedad del evento se verifican en el servidor; refresh rotativo, logout real, límites de solicitudes y claves privadas fuera de Git. La API usa un rol SQL sin permiso para modificar/borrar la evidencia de auditoría. El limitador entregado es por proceso, no distribuido.

Pruebas realizadas y evidencia

Automatización conservada

Grupo	Evidencia y alcance
Protocolo criptográfico	8 pruebas actuales: firmas, posesión, manipulación, eventos/zonas, revocación, transferencia y límites temporales para todos los segundos de una ventana
Sesiones del portal	5 pruebas sobre persistencia y renovación de sesión
API + PostgreSQL	Informe histórico con 18 escenarios aprobados; incluye concurrencia, precio, expiración, permisos y escaneo por protocolo
Portal en Chromium	Recorrido histórico de 18 pasos con API real, creación/configuración, reembolsos, auditoría y vistas de escritorio/móvil
Clon limpio	Informe de arranque y segunda inicialización sin duplicados, con volúmenes nuevos
Consultas SQL	EXPLAIN conservado de cuatro consultas, con justificación de índices

Fuentes: integración, carga original, portal, clon limpio, EXPLAIN y tests criptográficos.

Los informes históricos corresponden a las revisiones y fechas allí registradas. No se atribuyen automáticamente a cada commit posterior: el ajuste a 25 segundos tiene su propia verificación criptográfica y de compilación. Un test que simula dispositivos mediante HTTP no acredita cámara, SQLite ni modo avión en Android.

Carga repetida por el autor en GCP

El autor compartió en la sesión la salida real del script npm run test:load ejecutado contra la instalación de GCP:

Medición	Resultado observado
Compradores concurrentes	200
Ventas completadas	50
Respuestas esperadas de cupo agotado	150
Errores inesperados	0
Reserva p50 / p95	2.205,84 / 2.844,92 ms
Pago p50 / p95	1.912,45 / 2.088,16 ms
Estado del script	passed: true

La prueba autentica cuentas ya sembradas, crea un evento QA aislado con 50 entradas y lanza 200 solicitudes de reserva concurrentes, pagando las que consiguen cupo. También comprueba tickets, cargos, reservas e inventario mediante PostgreSQL y realiza una ráfaga de disponibilidad. No usa 200 correos creados manualmente ni un proveedor de correo.

La salida de GCP fue observada mediante captura aportada por el autor. El JSON completo de esa VM aún debe adjuntarse como evidencia independiente; no se reemplazó el JSON histórico por una transcripción. Las latencias son de una ejecución concreta, no un SLA ni una comparación controlada de plataformas.

Comprobaciones manuales Android y GCP

- Arranque de API, portal y PostgreSQL en una VM de GCP, con readiness correcto.

- Login del administrador en el portal y del asistente en Android. El diagnóstico dentro de la VM confirmó que la contraseña guardada autentica al administrador.

- Preparación de evento de ensayo con zona, fase, puertas y asignaciones de staff.

- Billetera con QR rotativo; renovación a 25 segundos confirmada por el autor tras actualizar y recargar Expo.

- Cámara virtual de otro emulador leyendo un QR y mostrando **Entrada válida**, visible en la captura compartida.

No se publican las capturas originales de la sesión: algunas incluyen credenciales, códigos de acceso o datos personales del escritorio. La cámara y QR válidos son evidencia parcial; no acreditan el recorrido offline completo.

Ejecutar y reproducir

Para el evaluador, desde un clon limpio con Docker y Compose v2:

```
docker compose up --build
```

Portal: http://localhost:8080. Readiness: http://localhost:3000/v1/readiness. Los jobs secrets e initialize terminan con código 0; los tres servicios persistentes deben quedar saludables. La contraseña demo se obtiene localmente siguiendo el README; no existe una contraseña pública común a todas las instalaciones.

Con Node 22+, desde la raíz:

```
npm ci --ignore-scripts
npm run build
npm test
npm run typecheck
npm run test:integration
npm run test:load
```

Los dos últimos comandos requieren la instalación Compose accesible al cliente Docker del mismo entorno. Crean datos QA; no apuntarlos accidentalmente a una instalación de negocio real. El informe de carga se guarda en docs/qa/load-results.json. Para GCP y Docker temporal véase la guía de operación.

Para Android: instrucciones de Expo, APK mediante EAS. Hay configuración de APK, no confirmación de un APK terminado o validado. Expo Go puede necesitar Metro para reabrir el bundle sin red; para probar arranque totalmente offline se requiere un build con bundle incluido.

Pendientes de entrega y límites conocidos

- Video/GIF obligatorio de dos escáneres offline y conflicto tras sincronización; no está entregado. El video completo de 5–12 minutos es un bonus distinto.

- Prueba móvil completa de persistencia tras reinicio, billetera offline, duplicado local, QR antiguo, transferencia/revocación y miles de intentos.

- Detector de incidentes por picos sospechosos de compra no implementado. Rate limiting no sustituye ese requisito.

- La vista de aforo muestra ingresos únicos sincronizados; no registra salidas ni representa ocupación física exacta.

- El listado de transferencias está limitado a 100 sin cursor; otros listados principales sí tienen cursor.

- La auditoría de dependencias histórica registró 35 avisos del workspace móvil/devtools. El filtro de producción API/portal dio cero avisos en esa medición; no equivale a monorepo sin vulnerabilidades.

- Publicar la revisión final en main, verificar visibilidad pública y documentar la versión exacta entregada. Los cambios se preparan en feat/ticketing-platform.

- HTTPS público en GCP, CI, Wallet, push, lista de espera y asientos no se presentan como implementados.

La demostración visual pendiente no se sustituye por este documento. El repositorio puede servir como portafolio con alcance y resultados transparentes, pero no se declara satisfecho todo el enunciado.